Project Guidelines

Full-Stack Application Implementation

Frontend: React | Backend: Node.js | Database with Generated Dummy Data

Submission deadline	Thursday, 18 June 2026
Main deliverable	A working full-stack application based on the provided User Journeys document
Frontend technology	React
Backend technology	Node.js
Project management	All work must be managed through the team GitHub repository
Team contribution rule	Each team member must have at least one meaningful commit

Students are required to design and implement both the frontend and backend of an application, following the User Journeys document that will be uploaded to the CMS. The final project should demonstrate a complete user flow, working data handling, and clear collaboration through GitHub.

1. Project Objective

The objective of this project is to build a functional full-stack web application that translates the provided user journeys into an implemented software product. Students should focus on creating a usable interface, a structured backend, and a database populated with realistic dummy data.

2. Source of Requirements

The User Journeys document uploaded to the CMS is the main reference for the application requirements. Students must read it carefully and use it to identify the required pages, actions, user flows, backend operations, and data needed by the application.

- The implementation must follow the described user journeys as closely as possible.
- Any missing details may be reasonably assumed, but the assumptions should be documented in the README file.
- The application should support the main user actions described in the journey, not only static screens.

3. Required Technologies

Component	Requirement	Expected Work
Frontend	React	Create the user interface, pages, forms, routing, and frontend logic needed for the user journeys.
Backend	Node.js	Implement APIs, business logic, validation, and communication with the database.
Database	Any suitable database	Design the required data structure and insert generated dummy data for testing and demonstration.
Version control	GitHub	Manage all code, commits, branches, and collaboration through the repository.

4. Functional Requirements

- Implement the main screens and user flows described in the User Journeys document.
- Create a React frontend that communicates with the backend through API requests.
- Create a Node.js backend that exposes the required endpoints for the frontend.
- Store and retrieve application data from the database.
- Use forms, buttons, navigation, and feedback messages where needed to support the user experience.
- Handle basic errors such as missing inputs, failed requests, or invalid data.

5. Dummy Data Requirement

Students must generate realistic dummy data and feed it into their database. The dummy data should be sufficient to demonstrate the application during testing and presentation.

- The data should match the entities required by the application, such as users, products, services, orders, bookings, posts, requests, or any other relevant records.

- The database should not be empty at submission time.
- The README file should explain how the dummy data was generated and how it can be inserted or reset.
- A seed script is recommended, for example a script that inserts sample records into the database.

6. AI Usage Policy

The use of AI tools is allowed. However, students must be transparent about how AI was used in the project.

- Students must submit the AI chatlog used during development.
- The chatlog should include prompts and answers that helped generate, debug, explain, or improve the project.
- Students remain responsible for understanding, testing, and explaining all submitted code.
- AI-generated code should be reviewed and adapted to fit the project requirements.

7. GitHub and Collaboration Requirements

All project work must be managed on the team GitHub repository. The repository should clearly show the progress and contribution of the team.

- Each team member must have at least one meaningful commit in the repository.
- Commits should have clear messages that describe what was changed.
- The repository should include both frontend and backend code.
- The repository should include a README file with setup and running instructions.
- Students are encouraged to use branches or pull requests, especially when working in teams.

8. Expected Repository Structure

Students may choose their own structure, but the following structure is recommended:

```
project-name/
  frontend/
    src/
    package.json
  backend
    /
    src/
    package.json
  database/
    schema files and/or seed scripts
  docs/
    AI chatlog
    assumptions or notes
  README.md
```

9. README Requirements

The README file should make it easy for the evaluator to understand, install, and run the project. It should include:

- Project name and short description.
- Team members and their roles or contributions.
- Technologies used.
- Setup instructions for the frontend and backend.
- Database setup instructions and how to insert dummy data.
- Environment variables, if needed.
- A list of implemented user journeys.
- Any assumptions made when the User Journeys document did not provide enough detail.

10. Submission Requirements

Required item	Description
GitHub repository link	The repository must contain the complete frontend, backend, database scripts, README, and project files.
Working application	The application should run locally and demonstrate the implemented user journeys.
Dummy data	The database must contain generated dummy data or include a clear seed script to populate it.
AI chatlog	Submit the chatlog showing how AI was used during development, if AI was used.
README	Include setup steps, run commands, project explanation, and assumptions.
Commit history	Each team member must have at least one meaningful commit.

11. Evaluation Criteria

Criterion	What will be checked
User journey implementation	The application follows the CMS User Journeys document and supports the main flows.
Frontend quality	The React interface is usable, organized, and connected to the backend.
Backend quality	The Node.js backend provides functional APIs, handles data correctly, and includes basic validation.
Database and dummy data	The database structure is suitable and contains enough dummy data for testing.
GitHub collaboration	The repository shows clear progress, meaningful commits, and at least one commit per member.
Documentation	The README, AI chatlog, and setup instructions are clear and complete.

12. Final Checklist

- The frontend is implemented using React.
- The backend is implemented using Node.js.
- The app follows the User Journeys document uploaded to the CMS.
- The frontend communicates with the backend through APIs.
- The database is created and populated with dummy data.
- The GitHub repository contains all project code and documentation.
- Each team member has at least one meaningful commit.
- The README explains how to install, run, and test the project.
- The AI chatlog is included if AI was used.
- The project is submitted before the deadline: Thursday, 18 June 2026.

Important: Projects submitted without a working repository, missing backend/frontend implementation, no dummy data, or no visible team contribution may lose marks.